



UNIVERSIDADE
DE ÉVORA

Trabalho 1 – Sorting algorithms

Programação paralela

Realizado por:

Tiago Pinto m68106

João Lóios m68290

Mestrado em Engenharia Informática

Descrição dos algoritmos

- **Merge sort:** Este algoritmo usa um conceito de divisão e conquista aproveitando-se da recursividade para o fazer. A ideia geral é dividir um array de tamanho n , em n partes, ficando assim com n arrays todos com um elemento, a partir do momento em que não é possível dividir mais os arrays o algoritmo começa a junta-los, refazendo-os com os mesmos elementos originais, mas reorganizando-os no processo. Por exemplo Se tivermos o array $[3,9,6,1]$, o algoritmo começaria dividindo este array em $[3,9]$ e $[6,1]$, e depois em $[3]$, $[9]$, $[6]$ e $[1]$, a partir daqui o algoritmo vai juntar o $[3]$ com o $[9]$ e o $[6]$ com o $[1]$, mas desta vez os arrays surgiriam assim, $[3,9]$ e $[1,6]$, e após repetir este ciclo outra vez, teríamos o seguinte resultado $[1,3,6,9]$. Conseguimos assim um algoritmo com complexidade temporal $O(n \log n)$.

Nota: É comum que este algoritmo seja implementado usando uma função com 2 arrays auxiliares, normalmente "right" e "left", na nossa implementação isto não acontece. Para otimizar o programa, usamos o facto de todos os merges serem feitos com arrays sequenciais, usando assim os índices de começo dos 2 array e o final do último como guia, assim como o array original mais um temporário para organizar os valores diretamente no array original.

- **Odd-even:** O algoritmo Odd-Even, tem o objetivo de ordenar um vetor de inteiros através de múltiplas fases de comparação e troca entre elementos adjacentes. O funcionamento baseia-se em duas fases alternadas:
 - **Fase par (Even):** São comparados os pares de índices $(0,1)$, $(2,3)$, $(4,5)$, ... Se o elemento da esquerda for maior do que o da direita, ocorre uma troca.
 - **Fase ímpar (Odd):** São comparados os pares $(1,2)$, $(3,4)$, $(5,6)$, ... Voltando a trocar os elementos quando necessário.

O processo repete-se até que nenhuma troca seja realizada numa iteração completa, o que indica que o vetor está ordenado. O algoritmo apresenta uma complexidade temporal de $O(n^2)$.

Estratégia de paralelismo

- **Merge sort:** O paralelismo deste algoritmo é muito intuitivo e relativamente fácil, pois o algoritmo já trabalha de forma divisória, tudo o que temos de fazer é, no começo deste, quando as primeiras divisões são feitas, especificamos que cada metade vai ser processada por sua thread, e repetimos este processo o número de vezes desejado.

- **Odd-even:** A paralelização foi implementada utilizando threads em Java, através da classe Thread. Cada fase – par ou ímpar – é dividida entre múltiplas threads independentes. Durante cada fase, o vetor é dividido em blocos, cada thread recebe um intervalo de índices e, dentro desse intervalo, executa comparações e trocas de forma independente. Como os pares comparados numa mesma fase não se sobrepõem, não existem conflitos de escrita entre threads.

Exemplo – Fase Par:

Thread 1: compara (0,1) e (2,3)

Thread 2: compara (4,5) e (6,7)

Depois de iniciar todas as threads de uma fase, o programa utiliza o método join() para garantir que todas terminem antes do início da fase seguinte, porque a fase ímpar depende do resultado da fase par e sem sincronização, haveria inconsistência nos dados.

Análise de performance

Para ambos os algoritmos decidimos testar com 3 arrays de tamanho diferentes, e com 4 números diferentes de threads, 1, 2, 8 e 16. Escolhemos estes números pois o PC onde foram feitos os testes tem 8 núcleos com 2 threads por núcleo, fazendo assim uma performance ótima com 16 threads. Todos os valores foram medidos 5 vezes, sendo o resultado uma média dessas 5 medições. Todas as medições foram feitas pelo código e em ambos os algoritmos o tempo contado foi somente o tempo que o algoritmo demora a correr, ignorando assim o tempo que o código demora a ler e escrever os ficheiros.

- **Merge sort:**

	100 000	1 000 000	4 000 000
1 thread	26.6 ms	144.6 ms	559.2 ms
2 threads	36.6 ms	131 ms	368.6 ms
8 threads	40.6 ms	164 ms	292.8 ms
16 threads	42.2 ms	158.8 ms	284.8 ms

- **Strong Scalability:** Conseguimos ver facilmente que para arrays menores não é de todo vantajoso dividir o problema em mais do que uma thread, por exemplo, com 100 000 elementos é maior o tempo que o computador leva a criar e juntar threads do que apenas a resolver o problema. À medida que o array aumenta, podemos ver que começa sim, a ser vantajoso fazer essa divisão, por exemplo, com 1 000 000 já vale a pena dividir em 2 threads, dando assim 500 000 para cada uma, e com 4 000 000, devido à grande dimensão do array, já compensa separar o problema no maior número de threads possível, nunca ultrapassando o limite teórico do computador. Temos assim que quanto

maior a dimensão do array de valores mais “strongly scalable” o programa fica.

- **Weak Scalability:** Aqui conseguimos também averiguar muito facilmente que o programa tem uma fraca “weak scalability” pois por exemplo, em nenhuma das divisões de threads para o array com 1 000 000 de elementos, o programa foi sequer comparável ao array de 100 000, assim, neste exemplo o array cresceu 10X e nem com 16 threads se verificou uma redução do tempo de execução. O mesmo acontece entre os 2 exemplos maiores, onde o array aumentou 4 vezes, e apesar deste aumento, em nenhum momento o algoritmo executou mais rápido.

- **Odd-even:**

	1000	10 000	100 000
1 thread	102.6 ms	975 ms	17.694 s
2 threads	148 ms	1.480 s	20.467 s
8 threads	500 ms	5.347 s	58.472 s
16 threads	1.013 s	11.186 s	1 min 57 s

Os resultados indicam que o aumento do número de threads prejudica o desempenho do programa. Tal acontece porque o algoritmo cria e sincroniza threads em todas as fases da execução, introduzindo assim uma sobrecarga elevada. Como cada thread executa poucas operações, o custo de gestão das threads acaba por superar o ganho proporcionado pelo paralelismo.

Além disso, o Odd-Even Sort possui complexidade $O(n^2)$, o que limita fortemente a sua escalabilidade.

- **Strong Scalability:** O speedup observado foi inferior a 1 em todos os casos. Isto significa que a versão paralela se revelou sempre mais lenta do que a versão sequencial.

- **Weak Scalability:** A weak scalability também foi fraca, já que o aumento simultâneo do tamanho do vetor e do número de threads provocou um crescimento significativo do tempo de execução.

Comparação da performance teórica e prática

- **Merge sort:** Visto que este algoritmo divide o array em partes iguais, era expectável que à medida que as threads duplicam, o tempo reduzisse para metade, ou perto disso, tendo em conta o tempo da criação e junção das threads. Na realidade podemos ver que não foi isso que

aconteceu, pois com arrays de tamanho pequenos o tempo de criação e de junção das threads acaba por ser maior do que o tempo de execução do próprio algoritmo. Podemos também notar que quanto maior o array, mais perto ficamos dos tempos teóricos, pois quando trabalhamos com um array maior, o tempo relacionado com as threads começa a ser cada vez mais insignificante. Segundo o padrão averiguado nestes dados, é de se prever que se aumentássemos a quantidade de elementos a organizar, o tempo de execução do algoritmo iria aproximar-se do teórico.

- **Odd-even:** Do ponto de vista teórico, seria expectável que o tempo de execução diminuísse com a utilização de mais threads, resultando num ganho de desempenho paralelo. Contudo, o que se verificou foi um slowdown, ou seja, a versão paralela apresentou pior desempenho do que a versão sequencial. As principais causas para este fenómeno são a criação excessiva de threads em cada fase do algoritmo, a sincronização frequente através do método `join()`, a escassa carga de trabalho atribuída a cada thread, a complexidade quadrática do algoritmo Odd-Even Sort ($O(n^2)$) e a sobrecarga adicional ao nível da memória e da cache. Esta degradação torna-se ainda mais perceptível com 8 e 16 threads, onde o custo associado à gestão das threads anula por completo qualquer potencial benefício do paralelismo.

Comparação dos 2 algoritmos

Decidimos realizar este trabalho com estes 2 algoritmos, pois ambos são diferentes em diversos aspetos.

- **Simplicidade X eficiência:** Em termos de complexidade o merge-sort apresenta uma complexidade temporal de $O(n \log n)$ já o odd-even $O(n^2)$, isto mostra imediatamente que, apesar do odd-even ser mais simples e fácil de se compreender e implementar o merge-sort acaba por ser mais eficiente, conseguindo organizar arrays de dimensões muito superiores numa menor quantidade de tempo.

- **Paralelização:** Visto que o merge sort, na sua teoria e implementação genérica já envolve a divisão do array em partes iguais, a paralelização deste algoritmo é então trivial e muito prática de ser implementada, aumentando a capacidade e eficiência do algoritmo. Já o odd even, apesar de também permitir paralelização, necessita de sincronização entre cada fase de execução e mantém uma complexidade de $O(n^2)$, o que limita o ganho de desempenho. Assim, o merge sort apresenta uma melhor escalabilidade e eficiência para arrays de maior dimensão.

Com isto concluímos que enquanto o odd-even é simples e prático de ser implementado perde para o merge sort no que toca a eficiência e capacidade de paralelizar.

Compilar e correr

Para compilar ambos os programas realizar: `javac (algorithm).java`, em que (algorithm) é MergeSort ou oddEven. Já para correr temos para o odd-even: `java oddEven -t nthreads < input_files/array_narray.txt`, em que narray é o número de elementos do array, que corresponda a um ficheiro existente e nthreads é o número de threads desejado desde que seja potencia de 2. Para o merge sort: `java MergeSort -t nthreads` e posteriormente quando solicitado escrever o nome do ficheiro, dentro da pasta `input_files/array_narray.txt`, em que narray e nthread respeitam os mesmos requerimentos algoritmo anterior.